Full-stack + agentic AI

A study map for building product-grade applications with an AI layer

This is a topic map, not a roadmap. Each section lists what the topic gives you, what to study at three depths, and the reasoning to carry forward. The goal is pattern-making: after this you should be able to look at any feature and know which layer it belongs to, and what can go wrong there.

The one idea underneath everything

Think of your product as a well-run shop and the AI as a fast, confident helper who never says "I don't know." Almost all engineering work is building the shop around that helper: a clear register, a guard at the door, a written task list, a locked cash drawer, a small correct file per customer, a time limit, and a nightly list of mistakes. A good shop works with an average helper. A bad shop breaks with the best one.

Layer	Shop equivalent	Fails as
Domain model	The register of stock and orders	Nothing downstream is reliable
Boundary layer	Guard at the door	Wrong tenant, duplicate work, abuse
Orchestration	Task written on paper	Runs die mid-way, no resume
Agent loop	The helper deciding next step	Loops, wrong tool, hallucination
Guards and tiers	Locked cash drawer	Irreversible damage from one bad step
Context	The small file per customer	Confident answers from stale data
Effects	Receipt written and filed	No audit trail, no rollback
Evals and traces	Nightly list of mistakes	Same bug returns forever

Part one — the foundation layers

These exist whether or not you add AI. Weakness here cannot be fixed by a better model.

1. Product and domain modelling

What it helps you do: Turn a fuzzy product idea into explicit entities, states and transitions that code and agents can act on.

Basic — Entities and relationships, state machines for core objects, user roles and permissions, primary user journeys.

Intermediate — Domain-driven design (bounded contexts, aggregates, domain events), event storming, designing for idempotency.

Advanced — Event sourcing, CQRS, modelling user intent as a first-class stored object separate from system action.

Logic to internalise: An agent can only act well on a system whose states are explicit. Fuzzy domain, fuzzy agent.

2. Backend and system design

What it helps you do: Build the pipe that carries work reliably, so slow and failure-prone AI steps do not take the product down.

Basic — REST and RPC, authentication (sessions, JWT, OAuth), validation at boundaries, background jobs, webhooks.

Intermediate — Queues and workers, retries with exponential backoff, rate limiting, caching, transactional outbox, multi-tenancy (row-level vs schema isolation).

Advanced — Durable execution engines (Temporal, Inngest, Trigger.dev), sagas with compensating actions, exactly-once vs at-least-once, backpressure.

Logic to internalise: An agent run is a long-lived distributed workflow, not an HTTP request. Design it that way from day one.

3. Data layer

What it helps you do: Store both operational truth and AI-derived data without letting the second corrupt the first.

Basic — Relational modelling, indexing, migrations, soft deletes, audit columns.

Intermediate — Postgres JSONB, full-text search, pgvector, row-level security, read replicas.

Advanced — Hybrid search (keyword + vector + reranking), change data capture feeding context, data versioning, separation of operational and derived data.

Logic to internalise: AI output is derived data. Store it with provenance (model, prompt version, input) so it can be regenerated, audited or rolled back.

Part two — the AI layer

This is the part that is genuinely new. It is also smaller than most people expect.

4. LLM fundamentals (engineering view)

What it helps you do: Use the model as a component with known limits instead of a magic box you hope behaves.

Basic — Tokens, context windows, temperature, system prompts, structured output with JSON schema, streaming.

Intermediate — Prompts as versioned code, few-shot vs instruction design, prompt caching, choosing small vs large models per task.

Advanced — Failure modes (hallucination, instruction drift, position bias in long context), model routing and fallback chains, when fine-tuning actually beats prompting.

Logic to internalise: The model is a probabilistic component inside a deterministic system. Your job is to shrink the surface where probability matters.

5. Agent architecture patterns

What it helps you do: Choose the least amount of autonomy that solves the problem, and know the named patterns when you need more.

Basic — Tool calling, the think-act-observe loop, single agent with a small tool set.

Intermediate — Workflow vs agent (fixed graph vs model-chosen path), routing, prompt chaining, parallelisation, orchestrator-worker, evaluator-optimiser.

Advanced — Multi-agent systems (supervisor, hierarchical, handoff), planning vs reactive agents, graph state machines, checkpointing and resumable runs, subagent context isolation.

Logic to internalise: Most shipped AI features are eighty percent fixed workflow and twenty percent agent. Add agency only where the path genuinely cannot be known in advance.

6. Tools and integrations

What it helps you do: Give the model hands that are safe, legible and hard to misuse.

Basic — Tool schema design: clear names, descriptions, typed parameters, required vs optional.

Intermediate — Model Context Protocol (MCP), shaping tool results for a model reader, pagination and truncation, error messages the model can recover from.

Advanced — Permission tiers (read / reversible write / irreversible), dry-run modes, tool composition, dynamic tool loading when the tool count grows.

Logic to internalise: A tool is an API written for a model, not a human. Bad tool design causes more agent failures than weak models do.

7. Context, memory and retrieval

What it helps you do: Control exactly what the model sees at each step, which is the single biggest lever on output quality.

Basic — RAG pipeline (chunk, embed, retrieve, generate), conversation history handling.

Intermediate — Chunking strategies, metadata filtering, reranking, query rewriting, summarising long histories.

Advanced — Context engineering per step, short-term vs long-term vs episodic memory, memory write policies, agentic retrieval, knowledge graphs.

Logic to internalise: Most 'the model is dumb' bugs are 'the context was wrong' bugs. Check the file before blaming the helper.

Part three — what makes it a product

The layers that separate a working demo from something people pay for and trust.

8. Evals and observability

What it helps you do: Know whether a change made the system better, instead of shipping on impressions.

Basic — Log every model call (input, output, latency, cost), review outputs by hand regularly.

Intermediate — Build eval sets from real usage, LLM-as-judge, regression tests for prompt changes, tracing (LangSmith, Langfuse, OpenTelemetry).

Advanced — Online evals in production, trajectory evaluation (was the path good, not only the answer), A/B testing prompts and models, drift detection.

Logic to internalise: Evals are to AI features what unit tests are to code. Every bug you fix becomes a permanent test case so it cannot return silently.

9. Security and safety for agentic systems

What it helps you do: Make sure a compromised or confused agent still cannot cause serious damage.

Basic — Secrets handling, never trusting model output as safe input to SQL, shell or HTML.

Intermediate — Prompt injection (direct and indirect through retrieved content), output validation, least-privilege tool access, PII handling.

Advanced — Policy evaluation per tool call, sandboxed code execution, the risky combination of private data plus untrusted input plus external action, audit trails, cost-abuse prevention.

Logic to internalise: Anything the model reads can try to instruct it. Do not fix this by teaching the model suspicion; fix it by locking the drawer.

10. Cost, latency and scale

What it helps you do: Keep unit economics viable, because in AI products every feature carries a per-use cost.

Basic — Token cost arithmetic, streaming for perceived speed, measuring cost per request.

Intermediate — Prompt caching, semantic caching, batching, cheap-model-first routing, async processing for non-urgent work.

Advanced — Per-tenant cost tracking and limits, speculative execution, precomputing AI outputs, cost per user action vs revenue per action.

Logic to internalise: Architecture and pricing become the same decision. If one run costs more than the action earns, the design is wrong, not the price.

11. UX for AI products

What it helps you do: Build trust by making the system's uncertainty and actions visible and correctable.

Basic — Streaming and loading states, showing sources, making AI output editable.

Intermediate — Human-in-the-loop approval for risky actions, confidence signalling, undo, graceful fallback when AI fails.

Advanced — Generative UI, background agents with notifications, progressive autonomy as trust grows, designing for trust calibration.

Logic to internalise: Users forgive mistakes they can see and fix. They do not forgive silent ones.

12. Production and business layer

What it helps you do: Ship safely, learn from usage, and build something competitors cannot copy by switching models.

Basic — Deployment, environment config, error monitoring.

Intermediate — Feature flags for AI features, gradual rollout, usage-based billing, adoption analytics.

Advanced — Data flywheels (usage improves context and evals which improve the product), compliance (India DPDP Act, GDPR), multi-provider strategy to avoid lock-in.

Logic to internalise: The moat is rarely the model. It is your data, your workflow integration and your distribution.

The eight gates — run these for every feature

This is the thinking habit the whole map is meant to produce. Ask these in order and stop at the first one that rules a design out.

1	**Can rules solve it?**	If yes, no AI. Use the model only for ambiguity, language and judgement.
2	**Is the path known in advance?**	Known means a workflow with LLM steps. Unknown means an agent loop.
3	**What is the blast radius?**	Reversible: let it run. Costly: confirm. Irreversible: human approval.
4	**Where does state live?**	If the answer is 'in the running process', redesign. Every step must resume from the database.
5	**What is the minimum context?**	Less context that is correct beats more context that is complete.
6	**How will I know it was wrong?**	If you cannot answer this, you cannot ship it. Trace and eval before the feature.
7	**What is the fallback?**	Every AI path needs a non-AI route: a form, a queue, a human.
8	**What does one run cost?**	Multiply by expected usage and compare against what the feature earns.

Diagnosing a failure

When an AI feature gives a wrong answer, resist the urge to change the model first. Check in this order: **(1) Context** — was the right information in the window, and was any of it stale? **(2) Tools** — did a tool return a confusing shape, or an unrecoverable error message? **(3) Prompt** — was the instruction ambiguous or the output schema too loose? **(4) Model** — only now consider a different or larger model. In practice most failures are context failures, and a bigger model would have cost more and fixed nothing.

Where to go deep

Agent patterns	Anthropic engineering blog, including the Building Effective Agents essay
System design	Designing Data-Intensive Applications (Kleppmann)
AI engineering	AI Engineering (Chip Huyen)
Evals	Hamel Husain's writing on evaluation and LLM-as-judge
Security	Simon Willison's posts on prompt injection; OWASP Top 10 for LLM applications
Durable workflows	Temporal and Inngest documentation; LangGraph docs for graph state and checkpointing

Retrieval	pgvector documentation and hybrid search write-ups from search vendors

Study order: the foundation layers first, the AI layer second, the product layers third. A weak pipe with a brilliant agent makes an impressive demo that collapses under real traffic. A solid pipe with an average agent is a product you can improve every week.